

Claire AI Repo Structure And Code Skeletons

****CLAIRE AI -- REPO STRUCTURE + CODE SKELETONS (Engineering-Ready Layout Medium-Wide Unicode-Only Export-Ready)****

1. Overview

This document defines:

the top-level repository structure

the module layout

the engine folders

the schemas folder

the API layer structure

the governance & safety modules

the code skeletons for each subsystem

This is the artifact that shows Claire is ready for engineering implementation.

2. Top-Level Repository Structure

```
claire-ai/ core/ semantic/ concept_graph/ deterministic_engine/  
hybrid_fusion/ ingestion/ tech/ market/ competitor/ regulatory/ engines/  
breakthrough/ scoring/ portfolio_creator/ clustering/ strategic/ safety/  
safety_matrix/ redline_detection/ compliance/ audit/ modes/ deterministic/  
connected/ hybrid/ api/ routes/ controllers/ middleware/ schemas/  
breakthrough_object.py portfolio_candidate_object.py portfolio_item.py  
signals.py scores.py clusters.py safety.py system/ logging/ audit/ health/  
metadata/ docs/ architecture/ api/ safety/ governance/
```

3. Code Skeletons (High-Level)

These are not full implementations -- they are the interfaces and class structures that acquirers expect.

3.1 Breakthrough Engine Skeleton

```
# engines/breakthrough/engine.py class BreakthroughEngine: def  
__init__(self, semantic_layer, fusion_layer=None): self.semantic_layer =  
semantic_layer self.fusion_layer = fusion_layer def
```

```

detect_breakthrough(self, input_data): semantic_vectors =
self.semantic_layer.process(input_data) fused_vectors =
self._maybe_fuse(semantic_vectors) return
self._generate_breakthrough_object(fused_vectors) def _maybe_fuse(self,
vectors): if self.fusion_layer: return self.fusion_layer.fuse(vectors)
return vectors def _generate_breakthrough_object(self, vectors): # returns
BreakthroughObject pass

```

3.2 Portfolio Creator Engine Skeleton

```

# engines/portfolio_creator/engine.py class PortfolioCreatorEngine: def
__init__(self, scoring_engine, clustering_engine): self.scoring_engine =
scoring_engine self.clustering_engine = clustering_engine def
create_portfolio_item(self, pco): technical =
self._generate_technical_package(pco) market =
self._generate_market_package(pco) strategic =
self._generate_strategic_package(pco) risk =
self._generate_risk_profile(pco) return PortfolioItem(
technical=technical, market=market, strategic=strategic, risk=risk,
clustering=self.clustering_engine.assign(pco) )

```

3.3 Safety Layer Skeleton

```

# safety/redline_detection/detector.py class RedlineDetector: def
__init__(self, safety_matrix): self.safety_matrix = safety_matrix def
evaluate(self, breakthrough_object): flags = [] for entry in
self.safety_matrix.entries: if
entry.trigger_condition(breakthrough_object): flags.append(entry) return
flags

```

3.4 Mode-Switching Controller Skeleton

```

# modes/controller.py class ModeController: def __init__(self,
safety_layer): self.safety_layer = safety_layer self.state = ModeState()
def set_mode(self, mode, reason, override=False): if not override:
self.safety_layer.validate_mode_switch(mode) self.state.current_mode =
mode self.state.last_switch = now() self.state.reason = reason

```

3.5 API Route Skeleton

```

# api/routes/breakthroughs.py from controllers.breakthroughs import
BreakthroughController router = Router()
@router.post("/breakthroughs/detect") def detect_breakthrough(request):
return BreakthroughController.detect(request)

```

3.6 Schema Skeleton (Example: Breakthrough Object)

```

# schemas/breakthrough_object.py class BreakthroughObject(BaseModel): id:
str title: str description: str semantic_vectors: dict signals: dict
scores: dict safety: dict status: str

```

4. Engineering-Ready Folder Purpose Summary

Folder	Purpose
core/	Deterministic Semantic Foundations
ingestion/	Live data ingestion pipelines
engines/	Breakthrough, scoring, portfolio engines
safety/	Safety matrix, redline detection, compliance
modes/	Mode switching, isolation, activation logic
api/	REST API layer
schemas/	All data models
system/	Logging, audit, health
docs/	All documentation

5. Why This Matters for Acquisition

This repo structure + skeletons:

proves engineering feasibility

shows clean modularity

demonstrates subsystem isolation

aligns with enterprise engineering standards

accelerates diligence

reduces integration risk

increases valuation